



Advanced Locking Mechanisms

Nikita Kutsokon



Postgres Locks

Table Locks

Row Locks

Dead Locks

Page Locks

Table Locks

Depending on what kind of work you're doing — reading, inserting, updating, deleting, or changing the table structure — PostgreSQL automatically applies **different types of table locks** to keep things safe and consistent.

```
--ACCESS SHARE (used by SELECT)  
SELECT * FROM employees;
```

```
--ROW EXCLUSIVE (used by INSERT, UPDATE, DELETE)  
UPDATE employees SET salary = 50000 WHERE id = 1;
```

ACCESS EXCLUSIVE

- The strongest, most restrictive table-level lock.
- Automatically acquired by commands that make big changes, like: **DROP TABLE**, **TRUNCATE**, **VACUUM FULL**, and many **ALTER TABLE** operations.
- Blocks all other locks — no other transactions can read or write the table while it's held.
- Ensures exclusive access to safely perform major changes.

```
DROP TABLE employees;
```

ACCESS SHARE

- Most basic, non-intrusive table-level lock.
- Automatically acquired when running **SELECT** or any read-only query.
- Allows concurrent reads by multiple users.
- Blocks only **ACCESS EXCLUSIVE** operations like: **DROP TABLE**, **TRUNCATE**, heavy **ALTER TABLE**
- Prevents destructive changes while reading.
- **ACCESS EXCLUSIVE** waits for all **ACCESS SHARE** locks to release.

```
--ACCESS SHARE (used by SELECT)  
SELECT * FROM employees;
```

EXCLUSIVE

- Used to allow only reads to run concurrently, but blocks most data changes.
- Acquired automatically by commands like: **REFRESH MATERIALIZED VIEW CONCURRENTLY**
- Prevents other transactions from making data modifications while still allowing reads.
- Conflicts with many other locks to ensure the data stays stable during the operation.

ROW SHARE

- Acquired automatically by queries that lock rows for update, like:
SELECT ... FOR UPDATE, SELECT ... FOR SHARE
- Allows multiple transactions to read and lock rows without blocking each other.
- Blocks only strict locks like **EXCLUSIVE** and **ACCESS EXCLUSIVE**.
- Used to prevent conflicts when modifying rows.

```
BEGIN;  
SELECT * FROM orders WHERE id = 1 FOR UPDATE;  
-- Locks the row with id=1 to prevent others from updating it|
```

ROW EXCLUSIVE

- Acquired automatically by commands that modify data, like: **INSERT, UPDATE, DELETE, MERGE**
- Prevents other transactions from acquiring locks that conflict with data changes.
- Stronger than **ROW SHARE**, because it blocks more lock types.

```
BEGIN;  
UPDATE employees SET salary = salary * 1.1 WHERE id = 5;  
-- Acquires ROW EXCLUSIVE lock on the table  
COMMIT;
```

SHARE ROW EXCLUSIVE

- Used to protect a table during certain schema changes and trigger creation.
- Acquired automatically by commands like: **CREATE TRIGGER**, some **ALTER TABLE** operations.
- Blocks concurrent data changes and some other conflicting locks.
- Only one session can hold this lock at a time (self-exclusive).
- Ensures safe modification of table structure without data conflicts.

SHARE

- Used to protect a table while creating an index (without concurrency).
- Acquired automatically by commands like: **CREATE INDEX** (without **CONCURRENTLY**).
- Prevents data changes (**INSERT**, **UPDATE**, **DELETE**) while the index is being built.
- Allows other read operations to continue.
- Blocks locks that would modify the table's data or structure.

```
CREATE INDEX idx_name ON employees (last_name);
```

SHARE UPDATE EXCLUSIVE

- Used to protect a table during certain maintenance tasks.
- Acquired automatically by commands like:
VACUUM (without FULL), **ANALYZE**, **CREATE INDEX CONCURRENTLY**, **REINDEX CONCURRENTLY**, and some **ALTER TABLE** operations.
- Prevents other conflicting locks that might cause schema changes or heavy modifications.
- Allows normal reads and writes to continue while maintenance runs.
- Blocks other locks that could interfere with these operations, like concurrent schema changes.

Examples

```
BEGIN;  
  LOCK TABLE accounts IN ACCESS EXCLUSIVE MODE;  
  -- Critical operation  
  UPDATE accounts SET balance = balance + 100 WHERE id = 1;  
COMMIT;  
  
-- This ensures absolute safety from concurrent access.  
--No other transaction can even run SELECT on accounts until you COMMIT
```

Table locks

Lock Mode	Common Commands	Allows Reads?	Allows Writes?	Used for
ACCESS SHARE	SELECT	✓	✗	Safe reading
ROW SHARE	SELECT FOR UPDATE	✓	✗	Reading + reserving rows for future updates
ROW EXCLUSIVE	INSERT UPDATE DELETE	✓	✓	Standart writes
SHARE UPDATE EXCLUSIVE	VACUUM, ANALYZE, CREATE INDEX	✓	✓	Maintenance that tolerates light usage
SHARE	CREATE INDEX (non-concurent)	✓	✗	Index creation
SHARE ROW EXCLUSIVE	CREATE TRIGGER, ALTER TABLE	✓	✗	Schema changes, one session at a time
EXCLUSIVE	REFRESH MATERIALIZED VIEW CONCURRENTLY	✓	✗	Safe read-only during important refresh
ACCESS EXCLUSIVE	DROP TABLE, TRUNCATE, ALTER	✗	✗	Major changes, full control

Row locks

PostgreSQL uses table locks and row locks to manage concurrent access. Table locks affect the whole table and are used for structural changes or bulk operations like `ALTER TABLE` or `TRUNCATE`. They reduce concurrency but ensure data integrity during large changes. Row locks, on the other hand, apply to individual rows and are used for fine-grained operations like updating a single record. This allows high concurrency and is ideal for systems where many transactions update different rows at the same time. Use table locks for broad changes, row locks for precise control.

FOR UPDATE

- Strongest row lock, blocks others from updating or deleting locked rows.
- Acquired by **DELETE**, **UPDATE** on unique-indexed columns, and **SELECT ... FOR UPDATE**.
- Normal **SELECT** can still read locked rows without blocking.

```
SELECT * FROM orders WHERE id = 123 FOR UPDATE;
```

FOR NO KEY UPDATE

- Weaker than FOR UPDATE, allows some concurrent locks.
- Acquired by UPDATE on columns without unique indexes.
- Blocks conflicting updates but allows SELECT FOR KEY SHARE

```
UPDATE orders SET quantity = 5 WHERE id = 123;  
-- Acquires FOR NO KEY UPDATE lock on the row
```

FOR SHARE

- Shared lock, allows multiple transactions to lock the same row for reading.
- Prevents data modifications (no UPDATE/DELETE allowed).
- Acquired by SELECT FOR SHARE.

```
SELECT * FROM orders WHERE status = 'pending' FOR SHARE;
```

FOR KEY SHARE

- Weakest row lock, similar to FOR SHARE but allows updates to non-unique columns.
- Acquired by SELECT FOR KEY SHARE.
- Used to block concurrent deletes or updates on unique key columns.

AVOIDING BLOCKED TRANSACTION

Here's a concise explanation of **NOWAIT** and **SKIP LOCKED** in PostgreSQL — these are options used with `SELECT ... FOR UPDATE` (or similar row-locking clauses) to control how to behave when a row is already locked by another transaction.

NOWAIT

If the row is already locked by another transaction, it immediately throws an error.

```
SELECT * FROM accounts WHERE id = 1 FOR UPDATE NOWAIT;
```

When to you need to know right away if a row is locked and don't want to wait

SKIP LOCKED

If the row is already locked, it skips it and moves on to the next available row.

```
SELECT * FROM jobs WHERE status = 'pending' FOR UPDATE SKIP LOCKED LIMIT 5;
```

You're processing items from a queue or pool, and don't care about locked rows

Page lock

In PostgreSQL, a page is a fixed-size 8KB block used to store rows in both tables and indexes. These pages live in the shared buffer pool, a region of shared memory accessible by all backend processes.

Page lock

PostgreSQL uses a process-based architecture, where each database connection spawns a separate OS process. These processes may compete for access to the same pages in memory, which introduces important concurrency concerns:

- Multiple readers of a page are allowed and pose no issue.
- Multiple writers to the same page must be serialized to prevent data corruption.
- PostgreSQL uses OS-style synchronization primitives like LWLocks (Lightweight Locks), mutexes, and semaphores.
- These locks coordinate concurrent access to ensure consistency and safety within the shared buffer pool.

What is a Deadlock

- **Transaction A** waits for a resource locked by **Transaction B**
- **Transaction B** waits for a resource locked by **Transaction A**

Detecting deadlocks

Enable logging of deadlocks by adding the following settings to your **postgresql.conf** file:

- **log_min_error_statement = 'ERROR'** - Tells PostgreSQL to log any SQL statement that causes an error, such as a deadlock or constraint violation
- **log_lock_waits = on** - Logs a message when a query has to wait too long to acquire a lock

```
-- log_min_error_statement = 'ERROR'  
-- log_lock_waits = on
```

Resolving Deadlock



Step 1

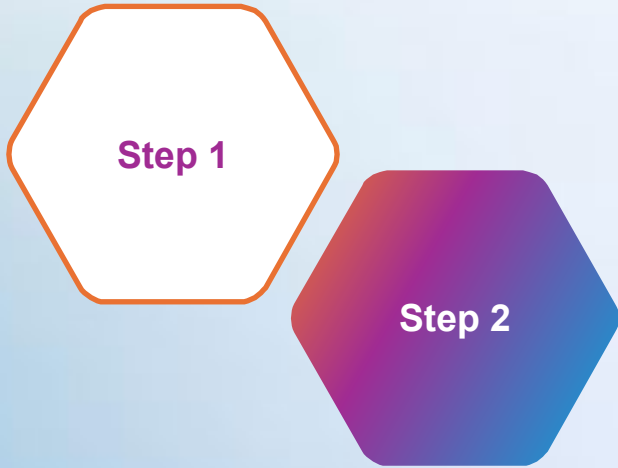
Identify the blocking transaction

This query returns a list of process IDs that are currently blocking your session in PostgreSQL

```
SELECT pg_blocking_pids(pid)
FROM pg_stat_activity
WHERE pid = pg_backend_pid();
```

- **pg_backend_pid()** – returns the PID of your current session (your database connection)
- **pg_stat_activity** – a system view that shows all active database sessions
- **pg_blocking_pids(pid)** – checks which other sessions are blocking the session with that PID

Resolving Deadlock



Terminate the blocking transaction

Forcefully ends the session that is blocking others with next query

```
SELECT pg_terminate_backend(blocking_pid);
```

pg_terminate_backend(pid) — forcefully kills the specified backend process in the database.

How to deal with deadlocks at the application level

Deadlocks happen when concurrent operations temporarily block each other. Because they are usually short-lived, retrying the transaction often works:

1. Detect deadlock errors in your database client or application.
2. Retry the operation a limited number of times, adding a short delay between attempts.
3. Keep transactions short to minimize conflict chances.

What are Advisory Locks?

Advisory Locks are user-controlled locks that you manage explicitly in your code. PostgreSQL does not enforce their use — you decide when and where to use them.

- **Session-level (`pg_advisory_lock`)**

Held until you explicitly release them or the session ends.

- **Transaction-level (`pg_advisory_xact_lock`)**

Held within a transaction (`BEGIN ... COMMIT`) and released automatically at the end of the transaction.

Examples

```
-- Process 1:  
SELECT pg_advisory_lock(12345);  
-- does some work with the resource  
  
-- Process 2 (concurrently):  
SELECT pg_advisory_lock(12345);  
-- will wait until Process 1 releases the lock  
  
-- To release the lock manually (Process 1 or any process holding it):  
SELECT pg_advisory_unlock(12345);
```

Examples

```
BEGIN;  
  
-- Acquire an advisory lock for the duration of the transaction (waits if locked)  
SELECT pg_advisory_xact_lock(12345);  
  
-- Your logic working with data here...  
  
COMMIT;  
-- After commit, the lock is automatically released
```

Thank you

- Author: Nikita Kutsokon
- My LinkedIn: <https://www.linkedin.com/in/nikita-kutsokon/>
- Jun 14, 2025
- [Join Codeus community in Discord](#)
- [Join Codeus community in LinkedIn](#)